SDG BLOCKCHAIN ACCELERATOR

Prototype (PoC) Report

ClimaFi – Results-Based Payments for Meteorological Data

In collaboration with UNDP Office of Procurement

1. Project Information

Project Name: Results-Based Payments for Meteorological Data (RBP-MetData)
Solution Maker: ClimaFi
Blockchain / Delivery Partner: Vaka Consulting
Challenge Owner: UNDP Office of Procurement (CCIT)
Report Version: Final PoC Submission v1.0

2. Executive Summary

This PoC demonstrates a blockchain-enabled system for **verifying meteorological data quality and linking it to a transparent results-based finance model**. The solution addresses a core institutional problem: countries and meteorological authorities often produce data that is valuable for climate resilience and global forecasting, but there is no sufficiently trusted, automated, and auditable mechanism to connect verified data delivery to financial incentives. The concept and delivery roadmap are grounded in the Cohort 2 challenge framing and pilot logic provided for the Office of Procurement / SOFF use case.

The PoC introduces four main capabilities:

1. **Automated data verification** against quality and timeliness rules aligned with GBON/WIS2-style requirements
2. **Tokenized proof of valid datasets**, where compliant submissions generate immutable "Data Receipt Tokens"
3. **Shadow payment simulation**, allowing stakeholders to test payout logic without moving real funds
4. **Audit-ready dashboards**, allowing providers and oversight entities to review compliance, proof history, and theoretical earnings

The implementation and debugging evidence show that the system was not only designed conceptually, but exercised through real backend, integration, and Cardano preview testnet workflows, including authentication, wallet management, minting, transaction validation, and API testing.

This is therefore not just a pitch-level prototype. It is a technically grounded PoC that demonstrates how a digital compliance and shadow-settlement layer could sit alongside meteorological data systems to support future results-based climate finance.

3. Challenge Context

Many countries, particularly least developed countries and small island developing states, remain trapped in an **"install-and-decay"** cycle for hydrometeorological infrastructure. Weather stations may be procured and deployed through donor or public investment, but their long-term maintenance is underfunded. As a result, infrastructure degrades, data quality drops, and the value of those systems erodes over time. This challenge is explicitly described in the provided Office of Procurement / SOFF roadmap, which frames the problem as the absence of a transparent verification layer linking data quality to funding.

In practical terms, the missing component is not simply more hardware. It is a trusted mechanism that can answer the following questions:

- Was the data delivered on time?
- Was the data complete and valid?
- Can an external funder independently verify the evidence?
- Can a payment or funding allocation be calculated based on objective proof?

This PoC responds to that missing layer.

4. PoC Objective

The objective of the PoC was to validate the technical and operational feasibility of a **digital audit and shadow accounting system** for meteorological data.

More specifically, the PoC aimed to prove that the solution could:

- ingest real or simulated station data,
- evaluate that data against agreed rules,
- generate a cryptographic proof of verified datasets,
- mint blockchain-based receipt records,
- simulate payout valuation logic,
- expose this information via dashboards for providers and oversight actors.

The "shadow" design is important. Rather than moving directly into live payments, the solution intentionally keeps the financial layer simulated during the PoC. That reflects stakeholder feedback captured in the roadmap: institutional trust, legal clarity, and operational validation must come before live settlement.

5. Solution Overview

5.1 High-Level Flow

The PoC follows the flow below:

Station / source data → ingestion layer → verification engine → approval/rejection → tokenized receipt → shadow valuation → dashboard / reporting

This architecture allows each step to remain separable and auditable:

- **Data ingestion** handles incoming weather observations
- **Verification** applies deterministic rules
- **Blockchain layer** records proof
- **Shadow logic** computes theoretical earnings
- **Dashboards** surface insights to stakeholders

5.2 Core Functional Modules

A. Data Ingestion Layer

This layer receives meteorological datasets through API or structured data ingestion pathways. In the long-term architecture described in the roadmap, this can sit alongside MQTT / REST style interfaces and ingest station or system-level data feeds.

Key responsibilities:

- normalize incoming payloads,
- validate schema,
- store submission records,
- prepare data for verification,
- preserve original submission integrity for hashing and audit.

B. Verification Engine

The verification engine is the heart of the PoC. It applies rules such as:

- timeliness,
- completeness,
- allowed value ranges,
- duplicate detection,
- structural consistency.

The roadmap also notes the importance of handling asynchronous validation, especially where rural connectivity causes delayed delivery. The model was therefore adapted so that valid delayed data can still be accepted within defined windows instead of being unfairly penalized.

C. Blockchain Receipt Layer

If a dataset passes verification, the system generates a cryptographic hash and mints a **Data Receipt Token** on Cardano. The token represents proof that a dataset met the agreed quality criteria at a given point in time. The planned metadata structure includes source identity, timestamp, and proof hash.

D. Shadow Settlement Logic

Instead of paying real stablecoins in the PoC, the system calculates what a payout would have been. This enables:

- testing valuation formulas,
- proving accounting consistency,
- building stakeholder confidence,
- avoiding regulatory exposure in the pilot phase.

E. Role-Based Dashboards

The solution includes role-sensitive views:

- **Provider / NMHS dashboard:** station performance, submission status, token history, theoretical earnings
- **Oversight / donor dashboard:** aggregated compliance, funding simulations, proof traceability, network health

6. Technical Architecture

6.1 Stack

The technical implementation described in the testing and debugging report includes the following stack: Python FastAPI backend, GraphQL, MongoDB, Cardano via PyCardano and Blockfrost, React frontend, AWS Lambda deployment, and supporting CI/CD tooling.

A consolidated architecture view is below:

Layer	Technology	Purpose
Frontend	React + TypeScript	Provider and oversight dashboards
API Layer	FastAPI + GraphQL	Business logic, data access, auth

Database	MongoDB	Submissions, users, verification records
Blockchain SDK	PyCardano	Wallets, minting, transaction building
Chain Access	Blockfrost	UTxOs, submission, chain context
Deployment	AWS Lambda + Serverless + Mangum	Serverless backend runtime
Testing	pytest, pytest-asyncio, httpx	Unit + integration coverage

6.2 Logical Components

```
┌──────────────────────────────┐
│ Meteorological Data Feed │
└──────────────────────────────┘
              │
              ▼

┌──────────────────────────────┐
│ Ingestion / Normalizer   │
│ - schema checks          │
│ - storage                │
└──────────────────────────────┘
              │
              ▼

┌──────────────────────────────┐
│ Verification Engine      │
│ - timeliness             │
│ - completeness           │
│ - anomaly checks         │
└──────────────────────────────┘
        │          │
  reject │          │ approve
        ▼          ▼

┌──────────────┐   ┌──────────────────────────┐
│ Fail Log   │   │ Hash + Token Minting │
└──────────────┘   └──────────────────────────┘
                          │
                          ▼

                 ┌──────────────────────────┐
                 │ Shadow Valuation     │
```

```
                  │ - payout simulation  │
                  └──────────────────────┘
                            │
                            ▼
                  ┌──────────────────────┐
                  │ Dashboards / Reports │
                  └──────────────────────┘
```

7. Data Model

```
A simplified submission model for the PoC could look like this:
{
 "submission_id": "subm_2026_01_05_0001",
 "provider_id": "tma_station_12",
 "station_id": "ST-012",
 "timestamp_utc": "2026-01-05T11:00:00Z",
 "payload_type": "weather_observation",
 "variables": {
   "temperature_c": 24.6,
   "humidity_pct": 75,
   "pressure_hpa": 1008.2,
   "wind_speed_ms": 4.8
 },
 "source_hash": "sha256:8ca0c1...",
 "verification_status": "approved",
 "token_minted": true,
 "shadow_value_usd": 12.50
}
```

This model makes it possible to trace:

- what was submitted,
- who submitted it,
- whether it passed,
- what hash was committed,
- whether a receipt token was minted,
- what shadow valuation was assigned.

8. Example Verification Logic

Below is a simplified Python-style example showing how a verification service might evaluate timeliness, completeness, and value ranges.

```
from datetime import datetime, timezone, timedelta
from hashlib import sha256
import json

REQUIRED_FIELDS = ["temperature_c", "humidity_pct", "pressure_hpa",
"wind_speed_ms"]

def verify_submission(submission: dict) -> dict:
   result = {
       "approved": False,
       "reasons": [],
       "hash": None
   }
```

1. Timeliness check

```
 ts = datetime.fromisoformat(submission["timestamp_utc"].replace("Z",
"+00:00"))
   now = datetime.now(timezone.utc)
   if now - ts > timedelta(hours=24):
       result["reasons"].append("Submission outside allowed delay window")
```

2. Completeness check

```
variables = submission.get("variables", {})
   missing = [field for field in REQUIRED_FIELDS if field not in variables]
   if missing:
       result["reasons"].append(f"Missing required fields: {missing}")
```

3. Range checks

```
  if "humidity_pct" in variables and not (0 <= variables["humidity_pct"] <=
100):
       result["reasons"].append("Humidity out of valid range")

   if "pressure_hpa" in variables and not (800 <= variables["pressure_hpa"]
```

```
<= 1200):
        result["reasons"].append("Pressure out of valid range")
```

4. Hash original canonical payload

```
 canonical = json.dumps(submission, sort_keys=True).encode("utf-8")
   result["hash"] = sha256(canonical).hexdigest()

   result["approved"] = len(result["reasons"]) == 0
   return result
```

This aligns with the PoC's emphasis on deterministic verification and cryptographic proofing before minting. The roadmap explicitly positions the verification engine as the logic checkpoint that decides whether a dataset should produce a receipt token or be rejected.

9. Example FastAPI Endpoint

The debugging report confirms a FastAPI backend with test coverage around authentication, CRUD, and workflow operations.
A simplified endpoint for submission handling could look like this:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Dict, Any

app = FastAPI()

class SubmissionIn(BaseModel):
   provider_id: str
   station_id: str
   timestamp_utc: str
   variables: Dict[str, Any]

@app.post("/submissions")
async def create_submission(payload: SubmissionIn):
   submission = payload.dict()

   verification = verify_submission(submission)

   record = {
```

```
        **submission,
        "verification_status": "approved" if verification["approved"] else
"rejected",
        "verification_reasons": verification["reasons"],
        "source_hash": verification["hash"]
    }

    # Example persistence layer call
    # db.submissions.insert_one(record)

    if not verification["approved"]:
        return {
            "message": "Submission rejected",
            "record": record
        }

    # Example minting call
    # token_tx = mint_receipt_token(record)

    return {
        "message": "Submission approved",
        "record": record,
        "minting_status": "pending"
    }
```

10. Example GraphQL Shape

The debugging report references GraphQL resolver logic and frontend integration paths. A reviewer-ready example of the API contract:

```
type Submission {
 id: ID!
 providerId: String!
 stationId: String!
 timestampUtc: String!
 verificationStatus: String!
 verificationReasons: [String!]!
 sourceHash: String!
 tokenMinted: Boolean!
 shadowValueUsd: Float
}
```

```
type Query {
 submissions(providerId: String, status: String): [Submission!]!
}

type Mutation {
 submitData(
   providerId: String!,
   stationId: String!,
   timestampUtc: String!,
   variables: JSON!
 ): Submission!
}
```

11. Blockchain Layer and Tokenization

11.1 Why Cardano

The roadmap positions Cardano as the selected infrastructure due to high assurance, deterministic behavior, and low transaction cost characteristics for this type of use case.

For this PoC, Cardano is used for:

- receipt token minting,
- cryptographic proof anchoring,
- shadow ledger logic,
- wallet-based identity and custody patterns.

11.2 Example Token Metadata

```
A receipt token can carry metadata such as:
{
 "provider_id": "TMA",
 "station_id": "ST-012",
 "timestamp_utc": "2026-01-05T11:00:00Z",
 "dataset_hash": "8ca0c1c94f9a...",
 "verification_rule_set": "gbon_v1",
 "status": "verified"
}
```

11.3 Example Minting Pseudocode

```
def mint_receipt_token(chain_context, issuer_wallet, recipient_address,
metadata):
    """
    Simplified pseudocode for minting a receipt token.
    Actual implementation would use PyCardano primitives, minting policies,
    min ADA calculations, metadata embedding, signing, and submission.
    """
    token_name =
f"receipt_{metadata['station_id']}_{metadata['timestamp_utc']}"

    tx = {
        "mint": {
            "asset_name": token_name,
            "quantity": 1
        },
        "metadata": metadata,
        "recipient": recipient_address
    }

    # Build -> sign -> submit using chain context and wallet keys
    return {
        "status": "submitted",
        "tx_hash": "example_tx_hash"
    }
```

This is consistent with the reported use of PyCardano and Blockfrost for wallet creation, token minting, distribution, and transaction verification on Cardano Preview.

12. Shadow Payment Logic

A key design principle of the PoC is that the financial rail remains simulated. This emerged directly from stakeholder feedback in the roadmap, which notes the need to build institutional confidence before introducing live crypto or public-sector payment rails.

A simplified shadow payout calculation may look like this:

```
def calculate_shadow_value(verification_status: str, data_quality_score:
float) -> float:
    base_value = 10.0
```

```
    if verification_status != "approved":
        return 0.0

    multiplier = min(max(data_quality_score, 0.0), 1.0)
    return round(base_value * multiplier, 2)
```

For example:

- approved + 0.95 quality score = 9.50
- rejected = 0.00

In real deployment, this logic could be expanded to include:

- frequency bonuses,
- coverage incentives,
- lateness penalties,
- donor-specific rate cards,
- performance windows by station or provider.

13. Security and Operational Design

The testing report surfaces several meaningful implementation issues and fixes, which strengthen the credibility of the PoC. These were not hypothetical; they reflect actual engineering hurdles encountered during implementation.

13.1 Issues Encountered

Examples include:

- Blockfrost authentication mismatch between mainnet and preview
- MongoDB IAM auth failures in Lambda
- JWT key decode failure
- minimum lovelace construction errors
- CORS failures
- missing user signatures in claim transactions

13.2 Why This Matters

These issues show that the team exercised the solution through realistic deployment conditions rather than stopping at wireframes or diagrams. The fixes also reveal operational maturity:

- environment-aware chain access,
- proper IAM-based DB auth,
- transaction safety,
- frontend/backend session handling,
- cross-origin readiness.

13.3 Remaining Hardening Areas

The same report also transparently notes what still remains before production-scale deployment:

- secrets management for seed storage,
- rate limiting,
- full frontend integration,
- automated blockchain CI tests,
- security audit before mainnet.

That transparency is a strength for reviewers. It makes clear which parts are proven and which parts still belong to pilot hardening.

14. Testing Strategy

The roadmap proposes a phased user and system testing approach, while the debugging report provides the implementation side of that testing. Together, they show a coherent validation plan.

14.1 Phases

1. Architecture & criteria validation
2. Pipeline and verification testing
3. Tokenization and registry validation
4. Dashboard and simulation testing
5. UAT and go/no-go assessment

14.2 Unit and Integration Coverage

The implemented backend used:

- pytest
- pytest-asyncio
- pytest-cov
- httpx

with markers covering auth, user management, transactions, admin, unit, and integration.

14.3 Example Test Case

```
import pytest

@pytest.mark.unit
def test_reject_submission_missing_field():
    submission = {
        "provider_id": "tma_01",
        "station_id": "ST-001",
        "timestamp_utc": "2026-01-05T11:00:00Z",
        "variables": {
            "temperature_c": 22.1,
            "humidity_pct": 70
        }
    }

    result = verify_submission(submission)
    assert result["approved"] is False
    assert any("Missing required fields" in reason for reason in
result["reasons"])
```

14.4 Example Integration Test Shape

```
import pytest
from httpx import AsyncClient

@pytest.mark.integration
async def test_submission_endpoint_rejects_invalid_payload(app):
    async with AsyncClient(app=app, base_url="http://test") as ac:
        response = await ac.post("/submissions", json={
            "provider_id": "tma_01",
            "station_id": "ST-001",
            "timestamp_utc": "2026-01-05T11:00:00Z",
            "variables": {
                "humidity_pct": 150
```

```
        }
    })

assert response.status_code == 200
body = response.json()
assert body["record"]["verification_status"] == "rejected"
```

15. UI / Dashboard Scope

The roadmap describes two principal dashboard perspectives: provider and donor/oversight.

15.1 Provider Dashboard

Key modules:

- station health
- submission timeline
- rejected data explanations
- receipt token history
- shadow earnings wallet

15.2 Oversight Dashboard

Key modules:

- compliance rate by provider/network
- token issuance trends
- verification pass/fail ratios
- payout simulations
- proof tracing by station, timestamp, or batc